

[Listing 10-6](#) has a function that accomplishes minimax.

```
import numpy as np
def minimax(max_or_min, tree):
    allscores = []
    for move_profile in tree:
        if move_profile[2] == []:
            allscores.append(move_profile[1][0] - move_profile[1][1])
        else:
            move, score = minimax((-1) * max_or_min, move_profile[2])
            allscores.append(score)
    newlist = [score * max_or_min for score in allscores]
    bestscore = max(newlist)
    bestmove = np.argmax(newlist)
    return(bestmove, max_or_min * bestscore)
```

[Listing 10-6](#): *A function that uses minimax to find the best move in a game tree*

Our `minimax()` function is relatively short. Most of it is a `for` loop that iterates over every move profile in our tree. If the move profile has no child moves, then we calculate the score associated with that move as the difference between our squares and our opponent's squares. If the move profile does have child moves, then we call `minimax()` on each child to get the score associated with each move. Then all we need to do is find the move associated with the maximum score.

We can call our `minimax()` function to find the best move to play in any turn in any in-progress game. Let's make sure everything is defined correctly before we call `minimax()`. First, let's define the game, and get all possible moves, using exactly the same code we used before:

```
allpossible = []

game = [[(1,2), (1,1)], [(3,3), (4,3)], [(1,5), (2,5)], [(1,2), (2,2)], [(2,2), (2,1)],
        [(1,1), (2,1)], \ [(3,4), (3,3)], [(3,4), (4,4)]]

gamesize = 5

for i in range(1, gamesize + 1):
    for j in range(2, gamesize + 1):
        allpossible.append([(i,j), (i,j - 1)])
```